

M02 / ASSESSMENT

M02 Module Revision Test

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
Rules	3
T01 - DE02-01 Database, table, row, column and query mental model	3
T02 - DE02-02 SELECT, aliases and LIMIT	3
T03 - DE02-03 WHERE and comparison operators	3
T04 - DE02-04 AND, OR, NOT and parentheses	3
T05 - DE02-05 ORDER BY, DISTINCT and result inspection	4
T06 - DE02-06 NULL, IS NULL and COALESCE	4
T07 - DE02-07 Arithmetic expressions and safe calculations	4
T08 - DE02-08 Aggregate functions	4
T09 - DE02-09 GROUP BY and grouped grain	4
T10 - DE02-10 HAVING after aggregation	4
T11 - DE02-11 CASE for business categories	5
T12 - DE02-12 String and date basics	5
T13 - DE02-13 Simple subqueries and IN/EXISTS awareness	5
T14 - DE02-14 Debugging and validation workflow	5

Read this first

MODULE REVISION TEST

Save your answers before opening the protected answer guide. No forced timer. First prove the competency untimed; later you may record interview-style time if useful. This test does not replace a protected Gate.

Coverage: 14 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

Rules

Work with learner book, review book, answer guide and AI solving closed. Save one answer/evidence item for every lesson. If a task needs a PC, write your prediction first, then run it. Mark PASS only when the result and your explanation agree.

T01 - DE02-01 | Database, table, row, column and query mental model

Use `course.retry_sales`. Request `order_id`, `quantity` and `unit_price` arranged by `order_id`. Explain why the order ID and quantity have different roles even though both may contain digit characters.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T02 - DE02-02 | SELECT, aliases and LIMIT

From `course.retry_sales`, return the last two IDs in descending order. Use the output label reference and retain item. DESC after `order_id` asks for descending order. Explain whether a second new query must use reference or `order_id` to select the original ID column.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T03 - DE02-03 | WHERE and comparison operators

From `course.retry_sales`, return rows ordered before November 2, 2026, with ID and date. Explain whether a November 2 record belongs in "before November 2."

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T04 - DE02-04 | AND, OR, NOT and parentheses

From `course.retry_sales`, return records that are not Paper and have quantity at least two. State which two rules each returned row satisfies.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T05 - DE02-05 | ORDER BY, DISTINCT and result inspection

Return distinct item and unit_price combinations from course.retry_sales. Then request the first two records under descending quantity and ascending order_id. Explain how the second key resolves the tied quantity of three.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T06 - DE02-06 | NULL, IS NULL and COALESCE

Use course.retry_sales. Return IDs whose customer_id is missing. Separately return IDs whose discount_pct is missing. Explain why those need not be the same record.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T07 - DE02-07 | Arithmetic expressions and safe calculations

Calculate gross amounts for course.retry_sales. Separately compute a return rate using 3 returned units and 0 sold units, preserving an undefined denominator with NULLIF. Explain why zero percent would misrepresent that case.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T08 - DE02-08 | Aggregate functions

Summarise course.retry_sales with total rows, known-payment rows, total quantity, gross total and mean sale amount. Explain why total quantity is not the denominator of the mean sale amount.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T09 - DE02-09 | GROUP BY and grouped grain

Group course.retry_sales by ordered_on. Return sale_rows, units and gross_total for each day. Reconcile all three measures to the source.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T10 - DE02-10 | HAVING after aggregation

From course.retry_sales, keep only groups with at least two sales records per category. Return category, count and gross total. Explain why COUNT(*) belongs in HAVING here.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T11 - DE02-11 | CASE for business categories

In `course.retry_sales`, define High as gross amount at least 75, Mid as at least 30 but below 75, and Low as below 30. Return ID and group, then identify the exact-boundary records that test your rule.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T12 - DE02-12 | String and date basics

For R203 in `course.retry_sales`, display the raw item, uppercase item, month number and date seven days later. Explain whether uppercase changes the stored value.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T13 - DE02-13 | Simple subqueries and IN/EXISTS awareness

Find `course.retry_sales` records at or above the `retry` table's average gross amount. State the threshold and explain whether EXISTS could replace that scalar threshold without changing the question.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T14 - DE02-14 | Debugging and validation workflow

For `course.retry_sales`, report gross amount by payment category with Paid, Unpaid and Unknown all preserved. Reconcile the category row counts and amounts to the source. You may repeat a CASE in GROUP BY; do not use a SELECT alias in WHERE. Explain what would go wrong if Unknown were put into Unpaid.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY